

RESUMEN TIPOS ABSTRACTOS Y ESTRUCTURA DE DATOS

Tipos Abstractos y Estructuras de Datos.....	1
Tipos Abstractos de Datos (TAD/TDA).....	2
Estructuras de Datos Estáticas.....	2
Estructuras de Datos Dinámicas	2
Organizaciones de Ficheros	5
Conceptos Fundamentales.....	5
Clasificación según su Uso o Vida.....	6
Tipos de Organización.....	6
Planificación de Disco y Almacenamiento.....	7
Algoritmos.....	7
Algoritmos de Búsqueda	8
Algoritmos de Ordenación	8
Algoritmos de Grafos.....	9
Formatos de Información y Ficheros	9

Tipos Abstractos y Estructuras de Datos

Las estructuras de datos son los sistemas o métodos de organización que permiten un almacenamiento eficiente de la información en la memoria principal. Se definen como un conjunto de variables de un determinado tipo agrupadas para representar un comportamiento lógico y facilitar la manipulación de datos según el problema y el algoritmo.

- **Gestión de Memoria:** Los punteros son la base de la gestión dinámica de memoria (uso de malloc y free en C), lo que permite que las estructuras crezcan en tiempo de ejecución.
- **Recursividad:** Las estructuras dinámicas (especialmente árboles) se manipulan habitualmente mediante recursión, y que

el sistema la simula internamente utilizando una pila (LIFO) para gestionar las llamadas.

Tipos Abstractos de Datos (TAD/TDA)

Un Tipo Abstracto de Datos es una estructura algebraica compuesta por una colección de datos y las operaciones asociadas a ellos (creación, transformación y análisis).

- **Abstracción y Ocultamiento:** Funcionan como una "caja negra"; el usuario conoce qué hace el TAD a través de su interfaz, pero no necesita saber cómo está implementado internamente.
- **Encapsulamiento:** Protege los datos prohibiendo el acceso directo a su implementación, lo que favorece la claridad y la reusabilidad del código.
- **Composición:** Un TAD puede estar compuesto por otros TADs; por ejemplo, se pueden construir pilas o colas utilizando arrays.

Estructuras de Datos Estáticas

Son aquellas cuyo tamaño en memoria se define en tiempo de compilación y permanece invariable durante la ejecución.

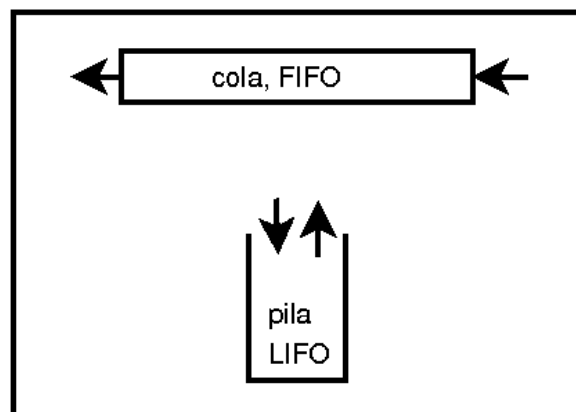
- **Arrays (Arreglos/Vectores):** Secuencias contiguas de elementos homogéneos (mismo tipo) accesibles mediante un índice. Pueden ser **monodimensionales (vectores)**, **bidimensionales (matrices)** o **multidimensionales (poliedros)**.
- **Registros:** Conjuntos de datos heterogéneos (pueden ser de distintos tipos) donde cada elemento se identifica por un nombre de campo.
- **Uniones:** Similares a los registros, pero todos sus campos comparten el mismo espacio físico en memoria, teniendo el tamaño del elemento más grande.
- **Punteros y Referencias:** Variables que almacenan una dirección de memoria. Mientras que un puntero es un valor numérico de la ubicación física, una referencia suele apuntar solo a objetos residentes en memoria.

Estructuras de Datos Dinámicas

Su tamaño puede variar durante la ejecución según las necesidades del programa.

- **Lineales o Secuenciales:**

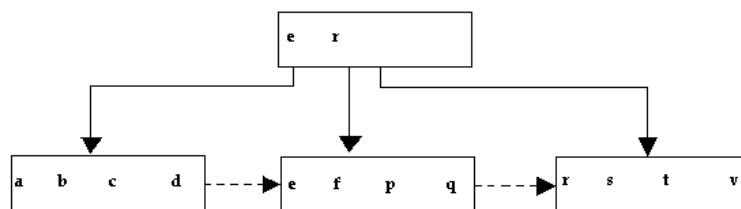
- **Listas:** Elementos ordenados consecutivamente. Pueden ser densas (la estructura define el siguiente, como en un array) o enlazadas (cada nodo tiene un puntero al siguiente). Existen variantes como las doblemente enlazadas (punteros a anterior y siguiente) y las circulares (el último apunta al primero).
 - **Listas Reorganizables:** Aquellas en las que, al acceder a un elemento, este se sitúa al principio para agilizar futuras consultas.
 - **Listas Circulares:** El último elemento tiene un enlace al primero, eliminando los nodos terminales nulos.
- **Pilas (LIFO):** El último en entrar es el primero en salir; utiliza operaciones push (apilar) y pop (desapilar).
- **Colas (FIFO):** El primero en entrar es el primero en salir; se inserta por el final y se extrae por el principio (frente).



- **No Lineales o Jerárquicas:**

- **Árboles:** Estructuras donde un nodo padre tiene nodos hijos. Destacan los binarios (máximo 2 hijos), los binarios de búsqueda y los AVL (balanceados donde la altura de subárboles difiere máximo en 1).
 - **Terminología precisa:**
 - **Grado de un nodo:** Número de descendientes directos (hijos).
 - **Grado de un árbol:** El máximo de los grados de todos sus nodos.

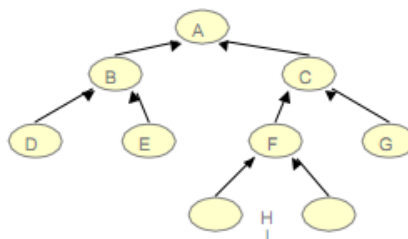
- **Altura vs. Profundidad:** La altura es el nivel máximo del árbol; la profundidad es la longitud del camino desde la raíz a un nodo específico.
- **Tipos de Árboles Binarios:**
 - **Perfecto:** Todos los niveles están llenos y las hojas están al mismo nivel.
 - **Completo:** Todos los niveles llenos excepto quizás el último, con nodos lo más a la izquierda posible.
 - **Árbol B+:** Variante donde los nodos hoja están enlazados en una lista, permitiendo un acceso secuencial eficiente además del directo, ideal para sistemas de ficheros.



- **Reconstrucción:** Para reconstruir un árbol binario de forma unívoca, es imprescindible contar con el recorrido Inorden más el Preorden o Postorden.

Recorridos

- *Preorder:* A B D E C F H I G
- *Inorder:* D B E A H F I C G
- *Postorder:* D E B H I F G C A



- Preorden: Raíz -> Izquierda -> Derecha.
- Inorden: Izquierda -> Raíz -> Derecha.
- Postorden: Izquierda -> Derecha -> Raíz.

- **Variantes de Árboles:**
 - **Árbol Lleno:** Cada nodo tiene 0 o 2 hijos.
 - **Árbol Perfecto:** Árbol lleno donde todas las hojas están al mismo nivel.
 - **Árbol Completo:** Niveles llenos excepto quizás el último, con nodos lo más a la izquierda posible.
- **Diferencia Árbol B vs B+:** El Árbol B los datos pueden estar en cualquier nivel, mientras que en el B+ todos los datos están en las hojas y estas se encuentran enlazadas, facilitando el acceso secuencial.
- **Grafos:** Conjunto de vértices (nodos) conectados por aristas (enlaces). Pueden ser dirigidos, no dirigidos, ponderados (con peso/coste) o completos.

Organizaciones de Ficheros

Un fichero o archivo es una colección de información con nombre que se almacena en memoria secundaria (disco), garantizando la persistencia de los datos tras finalizar los procesos.

Conceptos Fundamentales

- **Registro Lógico:** Colección de campos relacionados con una entidad (ej. los datos de un alumno).
- **Registro Físico (Bloque):** La cantidad mínima de datos que se transfiere en una operación de E/S.
- **Factor de Bloqueo:** Número de registros lógicos que caben en un bloque físico. Su incremento mejora el rendimiento al reducir el número de operaciones físicas de E/S, aunque aumenta el uso de la memoria intermedia (buffer).
- **Buffer:** Área de memoria principal que recibe temporalmente los datos del dispositivo externo.
- **Empaquetamiento de registros:** Puede ser Fuerte (aprovecha el sobrante de un bloque situando registros a caballo entre dos bloques contiguos) o Débil (no permite registros compartidos entre bloques)

Clasificación según su Uso o Vida

- **Permanentes:** Datos de larga duración. Se dividen en Maestros (estado actual), Constantes (datos fijos como códigos postales) e Históricos (datos pasados para reconstrucción).
- **Temporales:** Vida corta, usados para actualizar permanentes o como paso intermedio. Incluyen ficheros de Movimientos, de Maniobras (datos que no caben en RAM) y de Resultados.

Tipos de Organización

Determinan cómo se disponen los registros en el soporte físico:

1. **Secuencial:** Registros grabados uno tras otro. El acceso es siempre secuencial (hay que leer los anteriores para llegar a uno concreto). Puede ser encadenada si usa punteros para el orden lógico.
2. **Directa, Aleatoria o Relativa:** Permite el acceso inmediato a un registro mediante su clave o una función de cálculo (hashing), sin leer registros precedentes. Requiere soportes direccionables (discos).
3. **Indexada:** Combina un fichero de datos (normalmente secuencial) con una tabla de índices que asocia claves con direcciones físicas. Los índices pueden ser densos (uno por registro) o escasos (uno por segmento/bloque).

Organización Indexada (Métodos específicos):

- **ISAM:** Método de acceso secuencial indexado donde los registros se ordenan físicamente en disco; es rápido pero penaliza las inserciones por la gestión de índices.
- **VSAM:** Método de acceso de almacenamiento virtual (IBM) que soporta ficheros secuenciales (ESDS), de acceso directo (RRDS) e indexados (KSDS).

Algoritmos de Hashing (Transformación de claves):

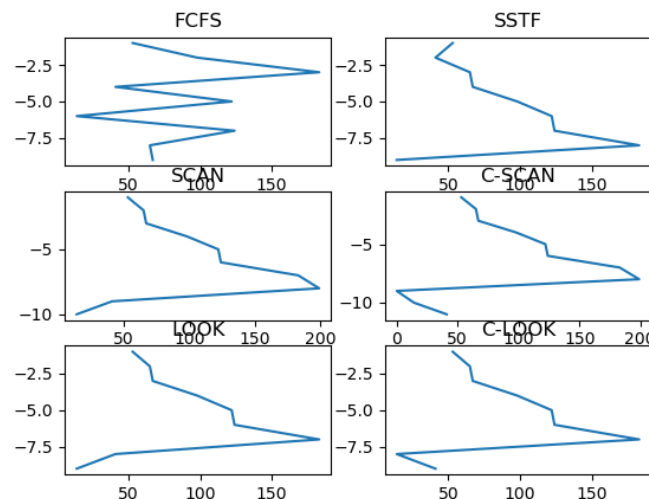
Una función hash ideal debe ser biyectiva y determinista, distribuyendo las claves uniformemente para minimizar colisiones.

- **Funciones comunes:** Aritmética modular (resto de división), Plegamiento (sumar trozos de la clave), Mitad del cuadrado (elevar y tomar dígitos centrales) y Truncamiento.

- **Tratamiento de colisiones:** Direcccionamiento abierto (buscar el siguiente hueco libre) o Encadenamiento (crear una lista enlazada en la dirección colisionada mediante punteros).

Planificación de Disco y Almacenamiento

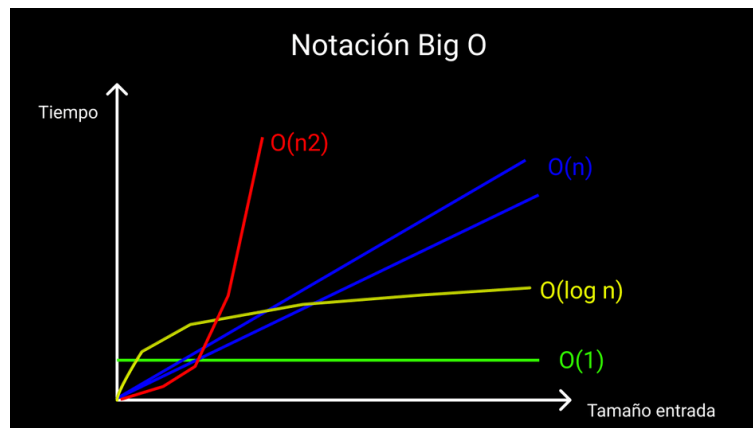
- **Algoritmos de planificación de brazo:**
 - **SSTF:** Atiende primero la petición más cercana a la posición actual (minimiza movimiento pero puede causar inanición).
 - **SCAN (Elevador):** El brazo recorre el disco de un extremo a otro atendiendo peticiones en su camino.
 - **C-SCAN:** Exploración circular; al llegar al centro, vuelve al exterior sin resolver peticiones para dar un tiempo medio constante.



- **Asignación de espacio:** Contigua (bloques adyacentes, causa fragmentación externa), Enlazada (lista enlazada de bloques, no hay fragmentación externa pero el acceso aleatorio es lento) e Indexada (nodos-i o inodos).

Algoritmos

Un algoritmo es una secuencia lógica, finita y precisa de instrucciones para resolver un problema. Su eficiencia se evalúa mediante la complejidad algorítmica (Notación O-grande).



Algoritmos de Búsqueda

- **Secuencial:** Recorre los elementos uno a uno. Complejidad $O(n)$.
- **Binaria o Dicotómica:** Divide el conjunto por la mitad sucesivamente. Requiere que los datos estén ordenados. Complejidad $O(\log n)$.
- **Hashing (Transformación de claves):** Aplica una función a la clave para obtener la dirección directa. Puede presentar colisiones (dos claves con igual índice).
- **Kosaraju / Tarjan:** Para hallar componentes fuertemente conectados en grafos dirigidos.
- **Ford-Fulkerson / Edmonds-Karp:** Para calcular el flujo máximo en una red.
- **Prim vs. Kruskal:** Ambos buscan el MST (Árbol de Expansión Mínima), pero Kruskal es generalmente más eficiente en grafos dispersos.

Algoritmos de Ordenación

- **Básicos ($O(n^2)$):** Selección (busca el mínimo y lo posiciona), Inserción (coloca cada elemento en su sitio en una sublista ya ordenada) y Burbuja.
- **Eficientes ($O(n \log n)$):**
 - **Quicksort:** Usa un pivote y técnica de "divide y vencerás".
 - **Mergesort:** Ordenación por mezcla recursiva.
 - **Heapsort:** Basado en la estructura de montículo o heap.
- **Ordenación Interna vs. Externa:** La interna se realiza en RAM (acceso aleatorio); la externa en memoria secundaria debido a que los datos no caben en RAM (restricciones de acceso).

- **Concepto de Estabilidad:** Un algoritmo es estable si mantiene el orden relativo de elementos con claves idénticas (ej. Mergesort es estable, Heapsort no lo es).
- **Algoritmos avanzados:**
 - **Shellsort:** Mejora de la inserción directa mediante "saltos" decrecientes.
 - **Radix sort:** Ordena por dígitos (LSD o MSD), limitado habitualmente a números enteros. Complejidad $O(k*n)$.

Algoritmos de Grafos

- **Recorridos:** Búsqueda en Anchura (BFS) (usa una cola) y Búsqueda en Profundidad (DFS) (usa una pila).
- **Rutas mínimas:** Dijkstra (pesos no negativos), Bellman-Ford (permite pesos negativos) y Floyd-Warshall (rutas entre todos los pares).
- **Árbol de Expansión Mínima (MST):** Algoritmos de Prim y Kruskal.

Formatos de Información y Ficheros

Los formatos definen la codificación interna de la información y suelen identificarse por la extensión del archivo.

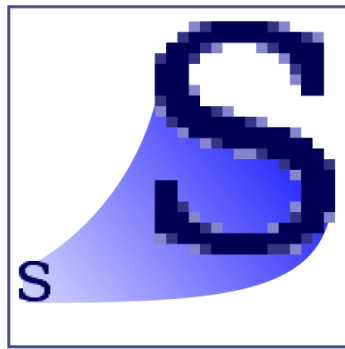
Número mágico: primeros bytes de un fichero que sirven para identificar su formato real.

- **Texto:** TXT (caracteres ASCII/ANSI) y RTF (enriquecido, legible en cualquier plataforma).
- **Texto Formateado/Compuesto:** DOCX (formato XML de Office), ODT (estándar abierto) y PDF (formato portátil de Adobe, basado en PostScript).

El **PDF** se define como un formato multiplataforma e independiente del hardware y sistema operativo. Entre sus capacidades técnicas generales destacan la protección mediante contraseñas, el uso de firmas digitales, la integración de metadatos (como autor y fecha) y el soporte para contenido interactivo como marcadores e hipervínculos.

- **PDF/A:** Es la versión diseñada para el archivado de documentos a largo plazo. Garantiza la reproducibilidad visual clara y requiere que todos los recursos (como fuentes) estén integrados en el archivo.

- **PDF/E:** Estándar internacional para documentación técnica e ingeniería. Es especialmente adecuado para dibujos tridimensionales (3D) de gran formato, permitiendo expandir o rotar elementos dentro del archivo.
- **PDF/UA:** Formato enfocado en la accesibilidad universal. Define requisitos precisos para que personas con discapacidad puedan utilizar los documentos, estructurando correctamente textos, imágenes y tablas.
- **PDF/X:** Desarrollado para el intercambio de documentos en preimpresión. Asegura que no se pierda información crítica de color o fuentes durante el proceso de diseño e impresión profesional.
- **PDF/VT:** Creado para requisitos de impresión de datos variables y transaccional. Se basa en PDF/X y busca procesar contenido recurrente de manera eficiente.
- **PDF/H:** Orientado a la industria sanitaria para el intercambio de datos médicos y almacenamiento de registros de pacientes.
- **PDF con capacidad de búsqueda:** Es el resultado de aplicar OCR (Reconocimiento Óptico de Caracteres) a documentos escaneados. Esto permite que el texto de una imagen sea reconocido, marcado y editado.
- **Formulario interactivo NOTA:** Es un tipo de PDF dinámico utilizado por la Agencia Española de Protección de Datos (AEPD) para presentaciones telemáticas; requiere Adobe Reader 7 o superior debido a sus funcionalidades de dinamismo.
- **PDF 417:** Se menciona como un código de barras bidimensional (nube de puntos) que se genera al imprimir formularios de notificación en papel para agilizar su lectura automatizada.
- **Imagen:**
 - **Raster (Mapa de bits):** JPEG (fotos, con pérdida), PNG (sin pérdida, soporta transparencia), GIF (animaciones, 256 colores) y TIFF (alta calidad).
 - **Vectoriales:** SVG (basado en XML, escalable), AI (Adobe Illustrator) y DWG/DXF (diseño CAD).

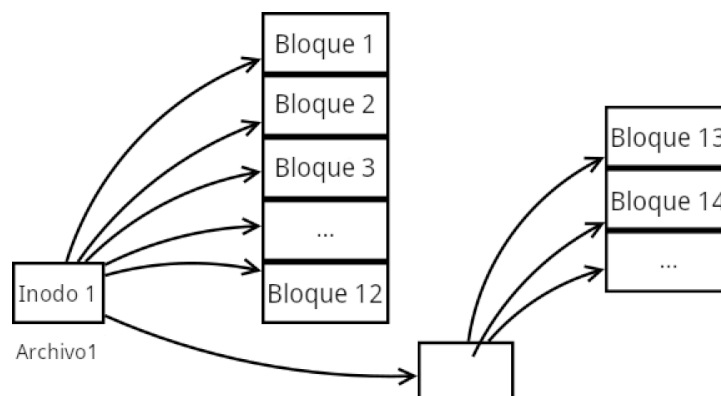


Raster
GIF, JPEG, PNG



Vector
SVG

- **Audio y Vídeo:** MP3, WAV (sin compresión), FLAC (sin pérdida) para audio; AVI, MP4, MOV para vídeo.
- **Comprimidos:** ZIP, RAR y TAR.GZ (común en Linux).
- **Intercambio y Web:** XML (estructurado, validación XSD), JSON (ligero) y YAML (formatos de texto legibles para humanos y máquinas), junto con HTML, CSS y JS para páginas web.
- **Sistemas de Ficheros (Formatos físicos):** FAT16/32, FAT32 (sin seguridad, límite de archivo 4GB), NTFS (Windows), EXT2/4 (Linux), HFS (macOS) y UFS (Unix).
- **Journaling (Registro por diario):** Mecanismo de EXT3, EXT4 y NTFS que anota los cambios antes de aplicarlos para garantizar la integridad tras un fallo.
- **Inodos (UNIX/Linux):** Estructura que contiene metadatos (dueño, permisos, fechas) y punteros a bloques de datos (directos, indirectos simples, dobles y triples). 12 directos, 1 indirecto simple, 1 indirecto doble y 1 indirecto triple.



- **Sistemas de archivos transaccionales:** ReFS (Windows Server) y EXT4, que utilizan estructuras de árboles B+ para mejorar la resiliencia y el rendimiento en grandes volúmenes.

- **Fragmentación:**
 - **Interna:** Espacio desaprovechado dentro de un bloque.
 - **Externa:** Huecos libres entre archivos incapaces de alojar nuevos procesos (típica de asignación contigua).